

Alumnex Connect: Unified Software Architecture & Agile SRS

1. Agile Project Charter & Requirements Matrix

1.1 Product Vision & Strategic Objectives

Alumnex Connect is an enterprise-grade networking and career acceleration ecosystem engineered to bridge the operational gap between current students, alumni, faculty, and institutional administrators. The system replaces broad, unverified professional networks with a verified, domain-specific community integrating real-time communication, automated AI career guidance, dynamic mentorship scheduling, and multi-platform developer benchmarking.

Key Metrics & OKRs:

- **Real-Time Communication Latency:** < 200 ms via persistent WebSocket dispatch.
- **Data Aggregation Speed:** Cached reads at 0 ms; API pipeline \$N\$ 4000 ms via `Promise.allSettled()` with `AbortController` timeouts.
- **Concurrency & Integrity:** 0 Oversold Event Bookings enforced via atomic `$expr` conditional filters.
- **Security Standard:** 100% Stateless XSS-Immune Auth using `httpOnly` secure cookies and cryptographic OTPs.

1.2 Granular Role-Based Access Control (RBAC) Matrix

Feature / Capability	Student Alumni		College	Admin
View Public Profiles	Full	Full	Full	Full
DevPulse Platform Linking	Full	Full	Read-Only	Full
Request Mentorship Session	Full	Restricted	No	Audit Only
Accept/Reject Mentees	No	Full	No	Override
Post Job / Internship	No	Full	Full	Full
Request Referral via DM	Full	Full	No	No
Create Campus Events	No	Full	Full	Full
Event RSVP Reservation	Full	Full	Full	Full
Forum Thread Creation	Full	Full	Full	Full
Global Moderation / Ban	No	No	Departmental Full System	

1.3 Product Backlog & Agile User Stories

Epic 1: Identity, Omni-Channel 2FA & Granular Access Control

US-01: Omni-Channel Two-Factor Registration

Story Points: 5 | Priority: MUST

Scenario: Successful student registration with secure OTP delivery
Given a student inputs valid email, password, and department details
When the submission is processed via `POST /api/auth/register`
Then a `User` document is created with `isVerified: false`
And an encrypted 6-digit OTP is delivered via SMTP Nodemailer within 30 seconds
And the client is directed to the OTP validation screen

US-02: Strict Cookie-Based Stateless Authentication

Story Points: 3 | Priority: MUST

Scenario: User login with `httpOnly` cookie dispatch
Given a registered user enters valid credentials
When `POST /api/auth/login` validates the `Bcrypt` hash
Then the server returns an `httpOnly` secure JWT token cookie
And the client state is hydrated with profile data without exposing JWT to JavaScript

Epic 2: Real-Time Event-Driven WebSocket Engine & Messaging

US-03: Stateful Direct Messaging with Database Fallback

Story Points: 8 | Priority: MUST

Scenario: Real-time message exchange between active users
Given User A is connected with `socketId` mapped in `onlineUsersMap`
When User A emits `message:send` targeted to User B
Then `Socket.IO` routes the payload to User B's active socket instantly
And the message document is asynchronously committed to MongoDB Atlas

US-04: Offline Push Fallback Notification

Story Points: 5 | Priority: SHOULD

Scenario: Message dispatch to an offline receiver
Given User B is not present in `onlineUsersMap`
When User A emits `message:send` to User B
Then the backend persists the message document
And dispatches a real-time `Notification` document alongside an asynchronous SMTP email alert

Epic 3: DevPulse Multi-Platform Ingestion, Timeout Resilience & Algorithmic Scoring

US-05: Parallel Scraper Pipeline with Non-Blocking Timeouts

Story Points: 8 | Priority: MUST

Scenario: Fetching multi-platform developer metrics

Given a user requests GET /api/dev-activity/:email

When node-cache encounters a cache miss

Then concurrent fetch requests are dispatched across GitHub, LeetCode, HackerRank, GFG, CodeChef, and Codeforces

And each request is guarded by an AbortController with a 4000ms hard ceiling

US-06: Bio-Verification Code Validation
Story Points: 5 | Priority: MUST

Scenario: Verifying platform account ownership

Given a user generates token ALUMNEX_VERIFY_XXXXXXX

When the user pastes the token into their external platform bio and triggers verification

Then the backend scraper confirms the token's presence in raw HTML/API payloads

And sets DevProfile.usernames[platform].isVerified = true

Epic 4: Mentorship Lifecycle, Dynamic Booking & Gamified Rewards

US-07: Structured Mentorship Workflow
Story Points: 5 | Priority: MUST

Scenario: Mentorship request acceptance lifecycle

Given a student initiates a request with goals to an alumnus

When the alumnus accepts via PUT /api/mentorship/request/:id/respond

Then a Mentorship document is instantiated with status: 'active'

And an automatic mutual follower relationship is committed to the database

US-08: MentorReward Leaderboard Accrual
Story Points: 5 | Priority: COULD

Scenario: Session logging and point allocation

Given an active Mentorship session reaches completed status

When the alumnus logs session hours in MentorshipSession

Then MentorReward.totalPoints increases by +50

And the alumnus's ranking updates across the global mentor leaderboard

Epic 5: Career Opportunities, Instant Referral Pings & Nested Forums

US-09: Real-Time Job Broadcast & 1-Click Referral DM
Story Points: 5 | Priority: MUST

Scenario: Student requests an instant job referral

Given a student views an active referral opportunity

When the student clicks "Request Referral"

Then an automated direct message payload containing resume links is delivered to the posting alumnus

And an application record is created with status: 'pending'

US-10: Recursive Forum Hierarchy with Author Safeguards
Story Points: 5 | Priority: MUST

Scenario: Nested thread replies and deletion bounds

Given an authenticated user views a forum post

When the user posts a reply up to 2 sub-document levels deep

Then the thread array updates in a single atomic document write

And only the verified author or admin can trigger deletion operations

Epic 6: Atomic Concurrency Controls & Global Event Management

US-11: Atomic Capacity Guard for Event Registrations
Story Points: 5 | Priority: MUST

Scenario: High-concurrency event RSVP

Given an event has a strict maximum capacity

When multiple concurrent requests invoke POST /api/events/:id/rsvp

Then MongoDB processes updates using \$expr: { \$lt: [{ \$size: "\$attendees" }, "\$maxAttendees"] }

And all requests beyond the limit return HTTP 400 without overselling seats

2. Complete Visual System Specifications

2.1 C4 Level 2: System Architecture Diagram

```

graph TD
    subgraph Client_Tier ["Client Tier (Frontend SPA)"]
        UI["React 18 Single Page App<br/>(Tailwind CSS, Glassmorphism UI)"]
        State["Context API Layer<br/>(Auth, Profile, SocketContext)"]
        Axios["React Query & Axios<br/>(httpOnly Interceptors)"]
        WSClient["Socket.IO Client Engine"]
        UI --> State
        State --> Axios
        State --> WSClient
    end

    subgraph Application_Tier ["Application Gateway Tier (Backend)"]
        Express["Express.js Server Router<br/>(Node.js v18+)"]
        AuthMiddleware["Security & RBAC Middleware<br/>(Helmet, Sanitization, Granular Guards)"]
        SocketServer["Socket.IO Stateful Server<br/>(In-Memory onlineUsersMap)"]
        DevPulseEngine["DevPulse Aggregator Service<br/>(Promise.allSettled & AbortController)"]
        MentorshipService["Mentorship & Job Service Layers"]

        Express --> AuthMiddleware
        AuthMiddleware --> DevPulseEngine
        AuthMiddleware --> MentorshipService
        Express <--> SocketServer
    end

    subgraph Caching_Tier ["In-Memory Caching & Performance"]
        NodeCache["node-cache<br/>(TTL: 2hr DevPulse / 30m Contests)"]
        RedisAdapter["Redis Pub/Sub Adapter<br/>(Socket.IO Multi-Node Sync)"]
        DevPulseEngine <--> NodeCache
        SocketServer <--> RedisAdapter
    end

    subgraph Storage_Tier ["Persistent Data Tier"]
        Mongo["MongoDB Atlas Database<br/>(23 Collections, Compound Indexes)"]
        MentorshipService <--> Mongo
        DevPulseEngine --> Mongo
        SocketServer --> Mongo
    end

    subgraph External_Tier ["Third-Party External Services"]
        Groq["Groq Cloud API<br/>(Llama 3 AI Mentor)"]
        Cloudinary["Cloudinary CDN<br/>(Media & Resume Storage)"]
        SMTP["Nodemailer Transport<br/>(SMTP OTP Service)"]
        Scrapers["Coding Platforms<br/>(GitHub, LeetCode, CF, HR, GFG, CC)"]
        Remotive["Remotive API<br/>(Remote Job Aggregation)"]

        DevPulseEngine --> Scrapers
        MentorshipService --> Remotive
        Express --> Groq
        Express --> Cloudinary
        Express --> SMTP
    end

    Axios -- "HTTP REST (Cookies)" --> Express
    WSClient -- "WebSocket (WSS)" --> SocketServer

```

2.2 Comprehensive Use Case Diagram

```

graph LR
    subgraph Actors
        S["Student Actor"]
        A["Alumni Actor"]
        AD["Admin Actor"]
        SYS["System Scheduler"]
    end

    subgraph Platform_Boundaries ["Alumnex Connect Boundary"]
        UC1["Register & Authenticate (OTP / 2FA)"]
        UC2["Link & Verify DevPulse Profiles"]
        UC3["Search Mentors & Book Sessions"]
        UC4["Accept / Reject Mentorship Requests"]
        UC5["Post Job & Referral Opportunities"]
        UC6["Apply for Job / 1-Click Referral DM"]
        UC7["Atomic Event RSVP"]
        UC8["Publish / Moderate Forum Threads"]
        UC9["Trigger Automated Cascade Deletion"]
        UC10["Calculate Global Leaderboards"]
    end

    S --> UC1
    S --> UC2
    S --> UC3
    S --> UC6
    S --> UC7
    S --> UC8

    A --> UC1
    A --> UC2
    A --> UC4
    A --> UC5
    A --> UC7
    A --> UC8

    AD --> UC1
    AD --> UC5
    AD --> UC7
    AD --> UC8
    AD --> UC9

    SYS --> UC10
    SYS --> UC2

```

2.3 Activity Diagram 1: DevPulse Parallel Aggregation Pipeline

```

flowchart TD
    Start([Start (Client Requests DevPulse Stats)]) --> CacheCheck{CacheCheck (Check node-cache<br/>TTL < 2 Hours?)}
    CacheCheck -- "Cache Hit (Valid)" --> ServeCache[ServeCache (Return Cached JSON Payload<br/>(0ms DB Latency))]
    ServeCache --> EndNode([EndNode (Render DevPulse UI)])

    CacheCheck -- "Cache Miss / Expired" --> InitControllers[InitControllers (Initialize AbortControllers<br/>(4000ms Hard Timeout))]
    InitControllers --> ParallelFetch[ParallelFetch (Execute Promise.allSettled() Concurrent Fetch)]

    subgraph Concurrent_Fetching ["Parallel Ingestion Pipeline"]
        F1[Fetch GitHub REST & Scraper]
        F2[Fetch LeetCode GraphQL]
        F3[Fetch HackerRank REST]
        F4[Fetch GeeksForGeeks API]
        F5[Fetch CodeChef Scraper]
        F6[Fetch Codeforces API]
    end

    ParallelFetch --> F1
    ParallelFetch --> F2
    ParallelFetch --> F3
    ParallelFetch --> F4
    ParallelFetch --> F5
    ParallelFetch --> F6

    F1 --> Aggregate[Aggregate (Consolidate Settled Promises<br/>(Filter Timeouts/Failures))]
    F2 --> Aggregate
    F3 --> Aggregate
    F4 --> Aggregate
    F5 --> Aggregate
    F6 --> Aggregate

    Aggregate --> MathScore[MathScore (Compute Alumnex Score Formula<br/>min(1000, LC + HR + GFG + GH + CF))]
    MathScore --> PersistDB[PersistDB (Persist Normalized Stats to MongoDB Atlas)]
    PersistDB --> PopulateCache[PopulateCache (Hydrate node-cache with 2-Hour TTL)]
    PopulateCache --> ReturnPayload[ReturnPayload (Return Fresh Compiled Stats Payload)]
    ReturnPayload --> EndNode

```

2.4 Activity Diagram 2: Atomic RSVP Concurrency Gate

```
Flowchart TD
    Start([Start]) -- "Student Clicks RSVP" --> RouteGuard{RouteGuard{Validate JWT Auth & Student Verification}}
    RouteGuard -- "Unauthorized" --> Err401["Return HTTP 401 / 403"]
    RouteGuard -- "Authorized" --> AtomicQuery[AtomicQuery["Execute findOneAndUpdate()<br/>Filter: _id == eventId<br/>$expr: $size(attendees) < maxAttendees"]]
    AtomicQuery --> MatchCheck{MatchCheck{Document Matched<br/>& Updated?}}
    MatchCheck -- "No (Capacity Exceeded)" --> Err400["Return HTTP 400<br/>'Capacity Reached / Closed'"]
    MatchCheck -- "Yes (Slot Available)" --> ExecPush["Mongoose Atomic Operation:<br/>$addToSet: attendees = studentId"]
    ExecPush --> SocketEmit["Socket.IO Server Action:<br/>Emit event:rsvp:success Broadcast"]
    SocketEmit --> UserNotif["Dispatch Toast & In-App Notification"]
    UserNotif --> Return200["Return HTTP 200 Success Payload"]
    Err400 --> End1([End])
    Err401 --> End2([End])
    Return200 --> End3([End])
```

2.5 Sequence Diagram 1: Stateful Direct Messaging & Offline Fallback

```
sequenceDiagram
    autonumber
    actor ClientA as User A (Sender Client)
    participant WS as Socket.IO Engine
    participant Mem as onlineUsersMap (Memory)
    participant DB as MongoDB Atlas Cluster
    actor ClientB as User B (Receiver Client)
    participant SMTP as Nodemailer SMTP Service

    ClientA->>WS: emit("message:send", {receiverId, content, mediaUrl})
    WS->>Mem: Check presence of receiverId in onlineUsersMap

    alt Receiver is ONLINE (Active Socket Found)
        Mem-->>WS: Return active target socketId (e.g., 'B456')
        WS->>ClientB: emit("message:receive", messagePayload)
        ClientB-->>WS: emit("message:ack")
        WS->>DB: Message.create({status: 'delivered', ...})
    else Receiver is OFFLINE (No Active Socket)
        Mem-->>WS: Return null
        WS->>DB: Message.create({status: 'pending', ...})
        WS->>DB: Notification.create({type: 'message', ...})
        WS->>SMTP: Trigger background Nodemailer alert email
        SMTP-->>ClientB: Deliver email notification ("New Unread Message")
    end

    WS-->>ClientA: emit("message:sent:ack", {timestamp, messageId})
```

2.6 Sequence Diagram 2: DevPulse Bi-Directional Platform Verification

```
sequenceDiagram
    autonumber
    actor Dev as Student / Developer
    participant UI as React Frontend UI
    participant API as Express API Router
    participant DB as MongoDB Atlas
    participant Ext as Target Platform (e.g., GitHub/LeetCode)

    Dev->>UI: Request platform verification code
    UI->>API: POST /api/dev-activity/generate-code
    API->>API: Generate cryptographic code "ALUMNEX_VERIFY_XXXX"
    API->>DB: DevProfile.updateOne({verificationCode, expiresAt})
    API-->>UI: Return verification code string

    Dev->>Ext: Paste verification code into public Profile Bio / Summary
    Dev->>UI: Click "Verify Platform Bio"
    UI->>API: POST /api/dev-activity/verify-platform {platform, username}

    API->>Ext: Scrape / Fetch public profile metadata
    Ext-->>API: Return raw bio HTML / JSON payload

    alt Token Found in Bio
        API->>DB: Set DevProfile.usernames[platform].isVerified = true
        API->>DB: Invalidate DevProfile.lastUpdated (Force Refresh)
        API-->>UI: Return HTTP 200 {verified: true}
        UI-->>Dev: Display Verified Green Shield Badge
    else Token Missing or Mismatched
        API-->>UI: Return HTTP 400 "Verification Token Not Found"
        UI-->>Dev: Display Error: "Please ensure code is saved in bio"
    end

    end
```

2.7 State Machine Diagram 1: Mentorship Lifecycle

```
stateDiagram-v2
    [*] --> Pending : Student submits mentorship request with goals

    Pending --> Rejected : Alumni rejects request
    Pending --> Accepted : Alumni accepts request

    Rejected --> [*] : Notification dispatched to student

    state Accepted {
        [*] --> ActiveConnection : Mutual follower created & Mentorship instantiated
        ActiveConnection --> InSession : MentorshipSession scheduled
        InSession --> ActiveConnection : Session completed & logged
    }

    Accepted --> Paused : Either party pauses relationship
    Paused --> Accepted : Resumed by mutual agreement

    Accepted --> Completed : Total target hours achieved
    Completed --> [*] : Calculate MentorRewards (+50 pts) & update Leaderboard
```

2.8 State Machine Diagram 2: Job Application Lifecycle

```
stateDiagram-v2
    [*] --> Pending : Student submits cover letter & resume

    Pending --> Reviewed : Alumni opens and evaluates application

    Reviewed --> Interviewing : Alumni initiates contact / DM referral
    Reviewed --> Rejected : Alumni updates status to Rejected

    Interviewing --> Accepted : Offer extended / referral cleared
    Interviewing --> Rejected : Candidate rejected post-review

    Accepted --> [*] : Notification & status badge updated
    Rejected --> [*] : System notification delivered
```

2.9 Unified Entity-Relationship Diagram (UML ERD)

```
erDiagram
    USER ||--o| DEVPROFILE : "owns (1:1)"
    USER ||--o{ JOB : "posts (1:M)"
    USER ||--o{ JOBAPPLICATION : "applies (1:M)"
    USER ||--o{ MENTORSHIP : "acts as mentor/mentee (1:M)"
    USER ||--o{ MESSAGE : "sends/receives (1:M)"
    USER ||--o{ FORUMPOST : "authors (1:M)"
    USER ||--o{ EVENT : "organizes/attends (1:M)"
    USER ||--o{ NOTIFICATION : "receives (1:M)"

    JOB ||--o{ JOBAPPLICATION : "contains (1:M)"
    MENTORSHIP ||--o{ MENTORSHIPSESSION : "schedules (1:M)"
    MENTORSHIP ||--o| MENTORREWARD : "accrues (1:1)"

    USER {
        ObjectId _id PK
        string name
        string email UK
        string password
        string role "student | alumni | college | admin"
        boolean isVerified
        string company
        int graduationYear
        stringArray skills
        ObjectIdArray connections
    }

    DEVPROFILE {
        ObjectId _id PK
        ObjectId user FK
        string email UK
        object usernames "github, leetcode, gfg, hackerrank, codechef, cf"
        object stats "cached multi-platform JSON metrics"
        int alumnexScore "Index: -1"
        date lastUpdated
        string verificationCode
    }

    JOB {
        ObjectId _id PK
        string title
        string company
        string type "Full-time | Internship | Referral"
        ObjectId postedBy FK
        boolean isActive "Index: 1"
        date deadline "Index: 1"
    }
```

```
JOBAPPLICATION {
  ObjectId _id PK
  ObjectId jobId FK
  ObjectId applicantId FK
  string resumeLink
  string status "pending | reviewed | accepted | rejected"
  date createdAt
}

MENTORSHIP {
  ObjectId _id PK
  ObjectId mentor FK
  ObjectId mentee FK
  string status "active | completed | paused | rejected"
  int totalHours
}

MENTORSHIPSESSION {
  ObjectId _id PK
  ObjectId mentorship FK
  date date
  int duration
  string notes
  string status
}

MENTORREWARD {
  ObjectId _id PK
  ObjectId mentor FK
  int totalPoints
  int rank
  stringArray badges
}

MESSAGE {
  ObjectId _id PK
  ObjectId sender FK
  ObjectId receiver FK
  ObjectId group FK
  string content
  string messageType
  date createdAt "Index: -1"
}

FORUMPOST {
  ObjectId _id PK
  ObjectId author FK
  string title
  string content
  string category
  objectArray replies "Recursive 2-level subdocuments"
}

EVENT {
  ObjectId _id PK
  ObjectId organizer FK
  string title
  date date
  int maxAttendees
  ObjectIdArray attendees
  boolean isOnline
}

NOTIFICATION {
  ObjectId _id PK
  ObjectId recipient FK
  ObjectId sender FK
  string type
  string message
  boolean isRead
}
```

3. Data Dictionary, IndexING Strategy & REST API Reference

3.1 Comprehensive Data Dictionary

User Schema Model

Field Name	BSON Type	Constraints & Validation	Description
_id	ObjectId	Primary Key, Auto-generated	Unique record identifier.
name	String	Required, trim: true, length \$lte\$ 2	Full legal name.
email	String	Required, Unique, Email Regex	Verification and login identity.
password	String	Required, Bcrypt Hash (Cost: 12)	Excluded from queries by default (select: false).
role	String	Enum: ['student', 'alumni', 'college', 'admin']	RBAC authorization driver.

isVerified	Boolean	Default: false	Controlled by SMTP OTP validation.
isApproved	Boolean	Default: false for college/alumni	Admin moderation flag.
connections	Array[ObjectId]	Ref: User model	Direct bi-directional network links.

DevProfile Schema Model

Field Name	BSON Type	Constraints & Validation	Description
_id	ObjectId	Primary Key, Auto-generated	Record identifier.
user	ObjectId	Required, Unique, Ref: User (1:1)	Linked user identity.
usernames	Object	Strict nested structure	Handles for GitHub, LeetCode, HackerRank, GFG, CodeChef, CF.
stats	Object	Mixed sub-document	Aggregated metrics, rankings, and badge URLs.
alumnexScore	Number	Min: 0, Max: 1000	Weighted developer score.
lastUpdated	Date	Timestamp format	Cache TTL tracking boundary (2 Hours).

Job & JobApplication Schema Models

Field Name	BSON Type	Constraints & Validation	Description
jobId	ObjectId	Ref: Job, Required	Target position.
applicantId	ObjectId	Ref: User, Required	Applying student.
resumeLink	String	Required, HTTPS URL validation	Cloudinary hosted PDF asset.
status	String	Enum: ['pending', 'reviewed', 'accepted', 'rejected']	Current application lifecycle state.

3.2 Production Indexing Strategy

Target Schema Compound Index Definition	Optimization Justification
User { email: 1, isVerified: 1 }	Eliminates collection scans for high-frequency login routes.
Message { sender: 1, receiver: 1, createdAt: -1 }	Accelerates reverse-chronologic chat pagination queries.
Job { postedBy: 1, isActive: 1 }	Facilitates instantaneous active opportunity listings.
DevProfile { alumnexScore: -1, user: 1 }	Supports index-covered global leaderboard generation.
Event { isActive: 1, deadline: 1 }	Filters expired events without scanning past documents.
JobApplication { jobId: 1, applicantId: 1 } (UNIQUE)	Unique index preventing double application submissions.

3.3 Comprehensive REST API Reference Catalog

Method Path	Auth Guard	Operational Target
POST /api/auth/register	Public	Hash pass & trigger OTP email.
POST /api/auth/verify-otp	Public	Validate OTP, set verified state.
POST /api/auth/login	Strict RateLimit (10 req/15m)	Validate hash, issue cookie JWT.
GET /api/users/:id	JWT Authenticated	Fetch user profile & network graph.
POST /api/users/:id/follow	JWT Authenticated	Toggle mutual network connections.
GET /api/jobs	JWT Authenticated	Merged DB + Remote API listings (Max 50).
POST /api/jobs	RBAC: Alumni / College / Admin	Post job, emit 'job:new' Socket.
POST /api/jobs/:id/apply	RBAC: Student	Submit PDF link & cover letter.
GET /api/dev-activity/:email	JWT Authenticated	Multi-platform 2hr cached metrics.
POST /api/dev-activity/verify	JWT Authenticated	Scrape platform bio for token validation.
POST /api/mentorship/request	RBAC: Student	Dispatch session request & notif.
PUT /api/mentorship/respond/:id	RBAC: Alumni	Accept/Reject request state machine.
POST /api/events/:id/rsvp	JWT Authenticated	Atomic \$expr capacity-guarded reservation.
GET /api/leaderboard	JWT Authenticated	High-speed sorted DevScore output.
POST /api/ai/chat	JWT Authenticated	Groq Llama 3 career assistant proxy.

4. Security Matrix, Defensive Architecture & DevOps Blueprint

4.1 Defensive Security Hardening Suite

Threat Vector	Mitigation Implementation Standard
Cross-Site Scripting (XSS)	Pure stateless httpOnly cookies for JWTs. JavaScript cannot access document.cookie. Strict CSP via Helmet.
NoSQL Injection Attacks	Sanitization via express-mongo-sanitize stripping \$ and . characters from incoming request payloads.
HTTP Parameter Pollution	hpp middleware protection against array parameter bugs.
Brute-Force / Enumeration	Dual-tier express-rate-limit: Global API limit + strict 10 requests / 15-minute limiter on /api/auth/*.
Cross-Origin Exploits	Explicit CORS allowlisting with credentials support (origin: process.env.FRONTEND_URL, credentials: true).

4.2 Data Integrity: Cascading Deletion Hooks

To eliminate orphaned subdocuments when a user exercises their right to account deletion, the Mongoose User schema registers a pre-hook middleware:

```
// backend/models/User.js - Cascade Deletion Hook
UserSchema.pre('deleteOne', { document: true, query: false }, async function(next) {
  const userId = this._id;
  await Promise.all([
    mongoose.model('DevProfile').deleteOne({ user: userId }),
    mongoose.model('JobApplication').deleteMany({ applicantId: userId }),
    mongoose.model('Mentorship').deleteMany({ $or: [{ mentor: userId }, { mentee: userId }] }),
    mongoose.model('ForumPost').deleteMany({ author: userId }),
    mongoose.model('Message').deleteMany({ $or: [{ sender: userId }, { receiver: userId }] }),
    mongoose.model('Notification').deleteMany({ $or: [{ recipient: userId }, { sender: userId }] })
  ]);
  next();
});
```


4.3 Production CI/CD Deployment Architecture

```
graph TD
    subgraph Development
        Dev[Developer Git Push] --> Master[master branch]
    end

    subgraph CI_Pipeline ["GitHub Actions & Automated Tests"]
        Master --> Tests["Jest Unit Tests<br/>(RBAC Middleware, Mentorship Load Balancer)"]
        Tests --> Cypress["Cypress E2E Mock Validation"]
    end

    subgraph CD_Deployment ["Render & Vercel Production Infrastructure"]
        Cypress --> Webhook1["Vercel CDN Webhook Trigger"]
        Cypress --> Webhook2["Render Backend Webhook Trigger"]

        Webhook1 --> ReactBuild["Build Static React 18 Assets<br/>(SPA Rewrites /index.html)"]
        Webhook2 --> NodeServer["Deploy Express + Socket.IO Server<br/>(pm2 daemonized runner)"]

        NodeServer <--> Atlas["MongoDB Atlas Cluster (TLS Encrypted)"]
        NodeServer <--> CloudinaryCDN["Cloudinary CDN (Signed Asset Streams)"]
    end
```

4.4 Automated Keep-Alive & Client Failover Pattern

- **Frontend Dynamic URL Resolution:** Client API instances monitor latency bounds; if requests to the primary Render backend time out (>10s on cold start), Axios intercepts the 504 and routes non-mutating reads to a mirror fallback gateway.
- **Scheduled Keep-Awake Self-Ping:** Render web services run a cron daemon triggering a lightweight endpoint (GET /) every 14 minutes, preventing node runtime deactivation during standard operation.